

EXPERIMENT-10

Q1. Write a C program to input time in seconds and convert it into days, hours, minutes, seconds format.

1. Problem Statement:

Write a C program to input time in seconds and convert it into days, hours, minutes, and seconds.

Input:

Total time in seconds.

Output:

The equivalent time in:

Days

Hours

Minutes

Seconds

Formula:

We know:

1 day = 86400 seconds

1 hour = 3600 seconds

1 minute = 60 seconds

Therefore:

days = totalSeconds / 86400

remainingSeconds = totalSeconds % 86400

hours = remainingSeconds / 3600

remainingSeconds = remainingSeconds % 3600

minutes = remainingSeconds / 60
seconds = remainingSeconds % 60

2. Algorithm:

Algorithm: Conversion of Seconds

1. Start.
2. Declare variables totalSeconds, days, hours, minutes, and seconds.
3. Input the total time in seconds.
4. Calculate days using: $\text{days} = \text{totalSeconds} / 86400$
5. Find the remaining seconds using: $\text{totalSeconds} = \text{totalSeconds} \% 86400$
6. Calculate hours using: $\text{hours} = \text{totalSeconds} / 3600$
7. Find the remaining seconds using: $\text{totalSeconds} = \text{totalSeconds} \% 3600$
8. Calculate minutes using: $\text{minutes} = \text{totalSeconds} / 60$
9. Calculate seconds using: $\text{seconds} = \text{totalSeconds} \% 60$
10. Display days, hours, minutes, and seconds.
11. Stop.

3. Pseudocode:

A.START

B.DECLARE totalSeconds, days, hours, minutes, seconds

C.INPUT totalSeconds

days $\leftarrow$ totalSeconds / 86400
totalSeconds $\leftarrow$ totalSeconds % 86400

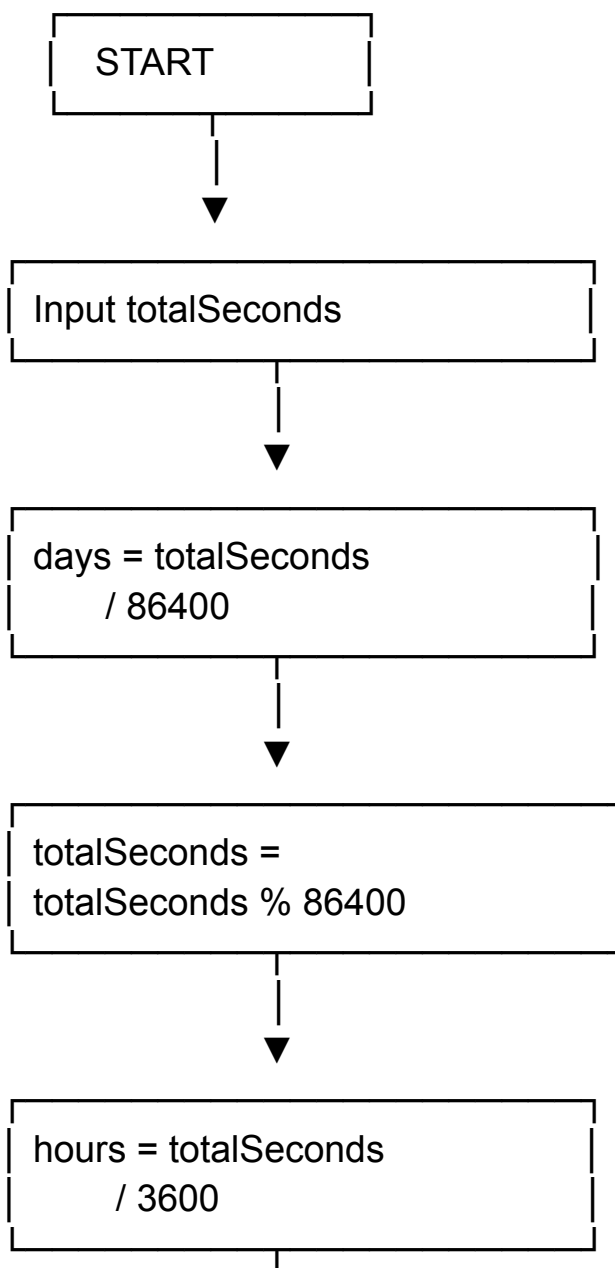
hours $\leftarrow$ totalSeconds / 3600
totalSeconds $\leftarrow$ totalSeconds % 3600

minutes $\leftarrow$ totalSeconds / 60
seconds $\leftarrow$ totalSeconds % 60

D.DISPLAY days, hours, minutes, seconds

E.STOP

4. Flowchart:





totalSeconds =
totalSeconds % 3600



minutes =
totalSeconds / 60



seconds =
totalSeconds % 60



Display days, hours,
minutes and seconds



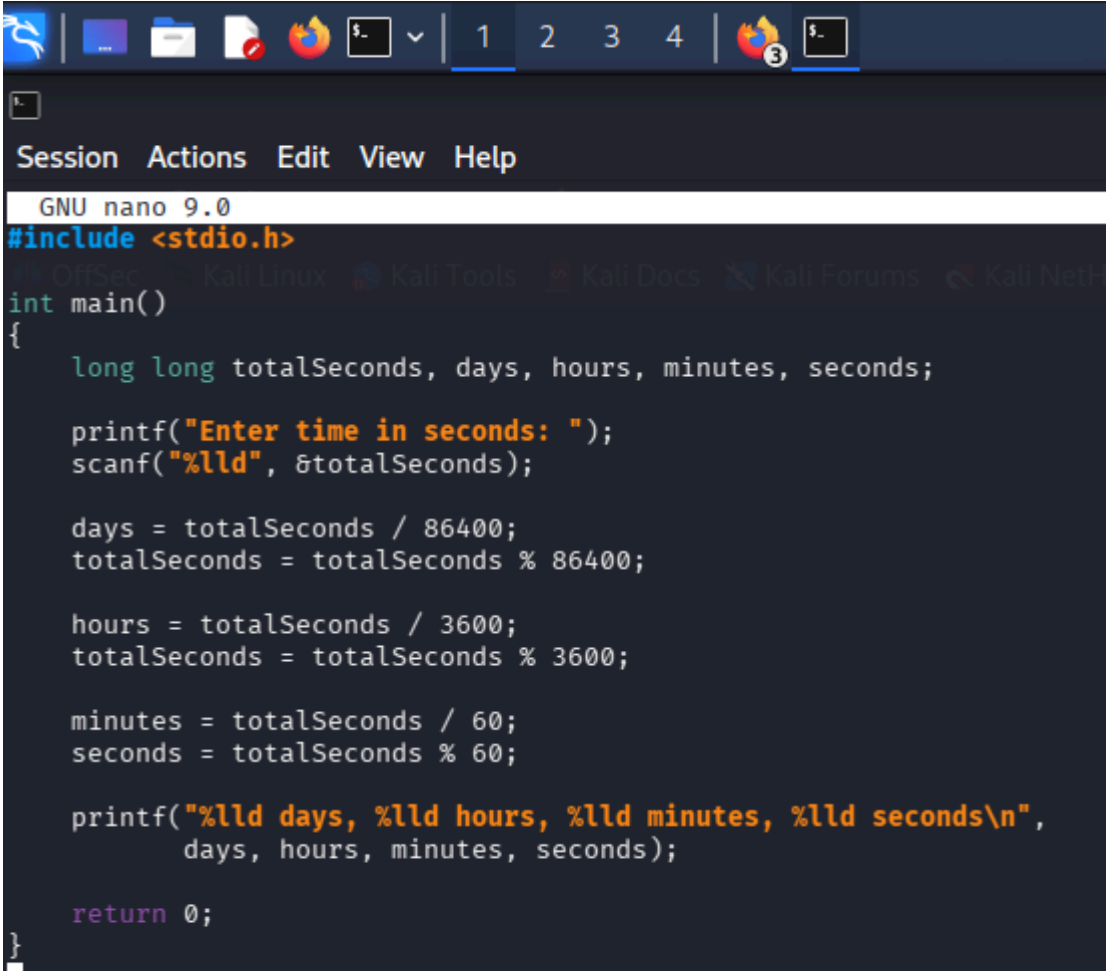
STOP

5. Test Case Table:

Test Case	Input (seconds)	Expected Output	Actual Output	Result
1	90061	1 day, 1 hour, 1 minute, 1 second	1 day, 1 hour, 1 minute, 1 second	Pass
2	86400	1 day, 0 hours, 0 minutes, 0 seconds	1 day, 0 hours, 0 minutes, 0 seconds	Pass
3	3661	0 days, 1 hour, 1 minute, 1 second	0 days, 1 hour, 1 minute, 1 second	Pass

The algorithm and flowchart give the expected result for all the prepared test cases.

6. Program, compilation and execution:



```
GNU nano 9.0
#include <stdio.h>

int main()
{
    long long totalSeconds, days, hours, minutes, seconds;

    printf("Enter time in seconds: ");
    scanf("%lld", &totalSeconds);

    days = totalSeconds / 86400;
    totalSeconds = totalSeconds % 86400;

    hours = totalSeconds / 3600;
    totalSeconds = totalSeconds % 3600;

    minutes = totalSeconds / 60;
    seconds = totalSeconds % 60;

    printf("%lld days, %lld hours, %lld minutes, %lld seconds\n",
           days, hours, minutes, seconds);

    return 0;
}
```

Result: The program is compiled successfully and passed all 3 prepared test cases.

Q2. Write a program to input a year and check whether and check whether it is a leap year or not.

1. Problem Statement:

Write a C program to input a year and check whether the entered year is a leap year or not.

Input:

A year, for example 2024.

Output:

Whether the given year is a leap year or not.

Leap Year Conditions:

A year is a leap year when:

It is divisible by 400, OR

It is divisible by 4 but not by 100.

For example:

2024 → Leap year

1900 → Not a leap year

2000 → Leap year

2. Algorithm:

Algorithm: Leap Year Check

1. Start.
2. Declare an integer variable year.
3. Input the year from the user.
4. Check whether $\text{year} \% 400 == 0$.
5. If yes, display "It is a leap year."
6. Otherwise, check whether $\text{year} \% 100 == 0$.
7. If yes, display "It is not a leap year."
8. Otherwise, check whether $\text{year} \% 4 == 0$.
9. If yes, display "It is a leap year."
10. Otherwise, display "It is not a leap year."
11. Stop.

3. Pseudocode:

START

DECLARE year

INPUT year

IF $\text{year} \% 400 = 0$ THEN
 DISPLAY "Leap Year"

ELSE IF $\text{year} \% 100 = 0$ THEN
 DISPLAY "Not a Leap Year"

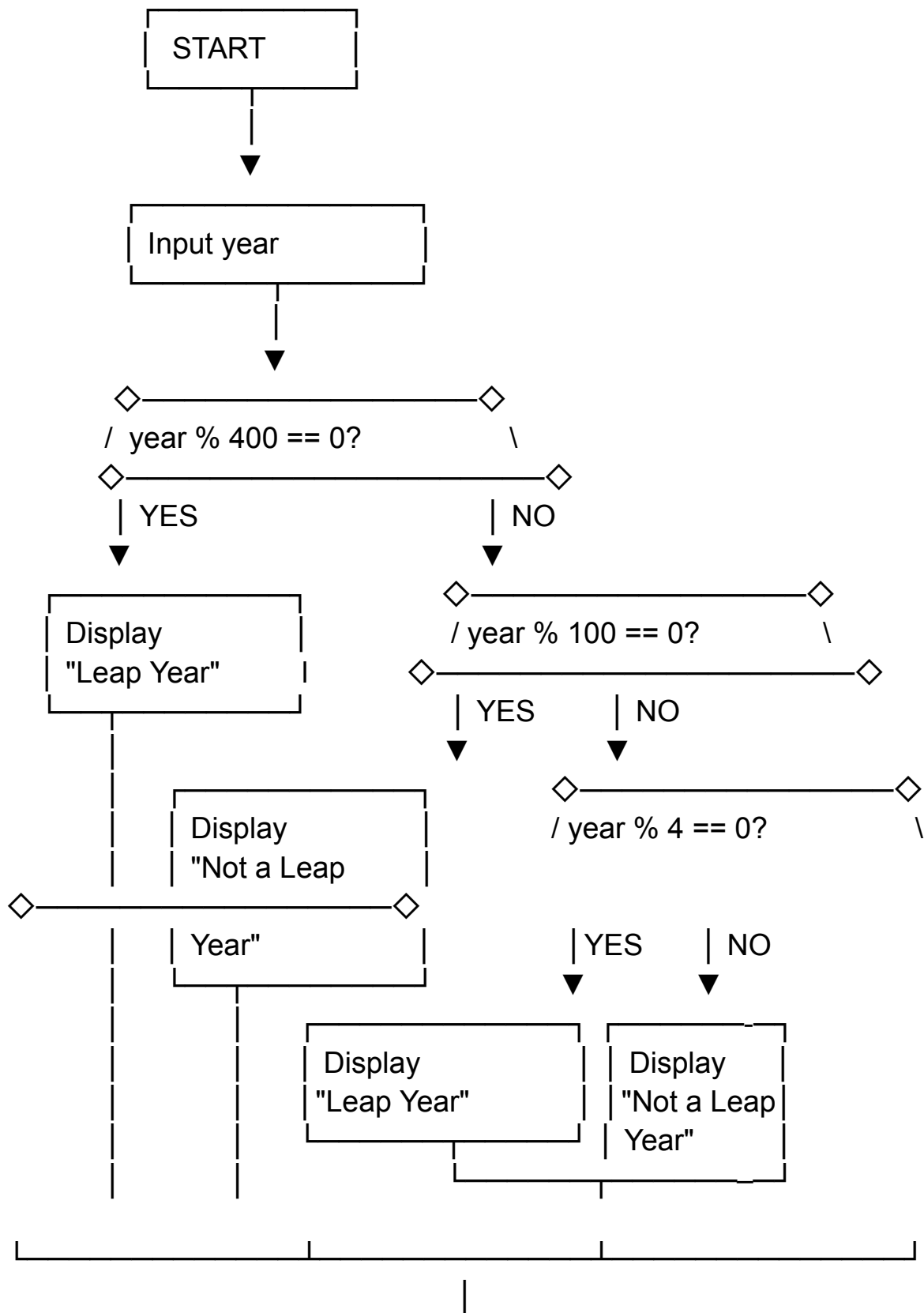
ELSE IF $\text{year} \% 4 = 0$ THEN
 DISPLAY "Leap Year"

ELSE
 DISPLAY "Not a Leap Year"

END IF

STOP

4. Flowchart:





5. Test Case Table:

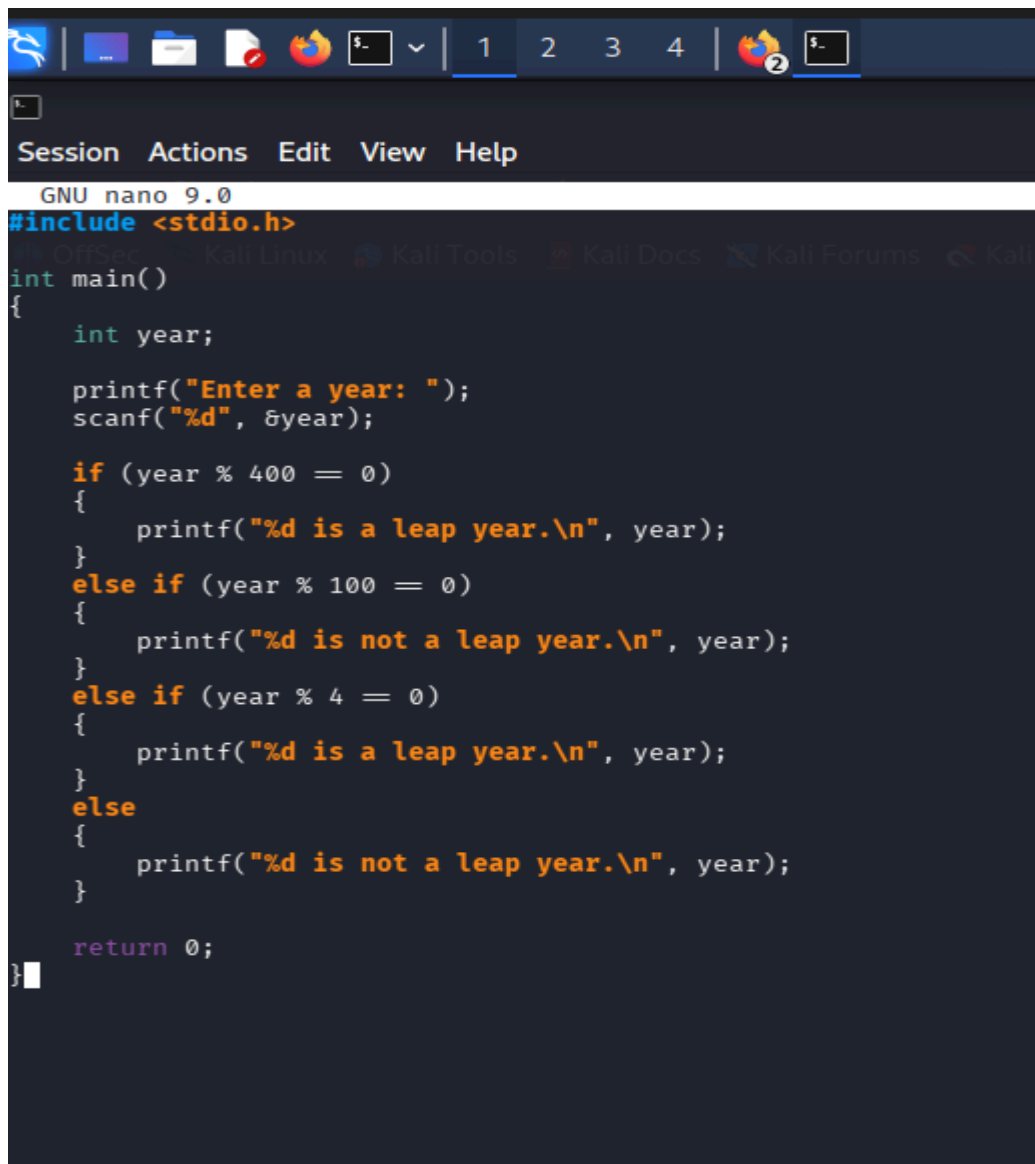
Test Case	Input Year	Expected Output	Actual Output	Result
1	2024	2024 is a leap year.	2024 is a leap year.	Pass
2	1900	1900 is not a leap year.	1900 is not a leap year.	Pass
3	2025	2025 is not a leap year.	2025 is not a leap year.	Pass

These cases are useful because they also test the special century-year rules.

6. Program, compilation and execution:

Result:

The program is compiled successfully and passed all 3 prepared tests.



```
GNU nano 9.0
#include <stdio.h>
int main()
{
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if (year % 400 == 0)
    {
        printf("%d is a leap year.\n", year);
    }
    else if (year % 100 == 0)
    {
        printf("%d is not a leap year.\n", year);
    }
    else if (year % 4 == 0)
    {
        printf("%d is a leap year.\n", year);
    }
    else
    {
        printf("%d is not a leap year.\n", year);
    }

    return 0;
}
```

```
(kali㉿kali)-[~]
$ nano year.c

(kali㉿kali)-[~]
$ gcc year.c -o year

(kali㉿kali)-[~]
$ ./year
Enter a year: 2024
2024 is a leap year.

(kali㉿kali)-[~]
$ ./year
Enter a year: 1900
1900 is not a leap year.

(kali㉿kali)-[~]
$ ./year
Enter a year: 2025
2025 is not a leap year.

(kali㉿kali)-[~]
$
```

NAME: TAANISH AGARWAL
BATCH: B-15
SAP ID: 590034662